

Dockerfile

第3讲

目标

从基本结构到镜像构建实战

- 基本结构（难点）
- 常用指令（重点）
- 实践教学：docker build 构建镜像

什么是 Dockerfile

一个纯文本文件，包含一系列指令，Docker 按顺序逐条执行，自动构建出镜像。

```
docker build -t myapp:1.0 .    # 读取当前目录的 Dockerfile，构建镜像
```



两大用途

-  对当前应用的描述 — 清晰记录应用及其依赖
-  指导 Docker 完成容器化 — 将应用打包为镜像

实现开发与部署的无缝切换，要像重视代码一样重视它，并纳入版本控制

命名与书写规则

命名与位置

- 文件名必须是 `Dockerfile`（大写 D），不能写成 `dockerfile` / `Docker file`
- 放在构建上下文根目录：`docker build` 会把整个目录发给守护进程，`COPY` 只能取这个范围内的文件

书写规则

规则	说明
注释	以 <code>#</code> 开头的行为注释行
指令格式	<code>INSTRUCTION argument</code> ，每行一条
大小写	不区分大小写，惯例 全大写
执行顺序	从上到下逐条执行

基本结构：四个组成部分

```
# ① 基础镜像（必须是第一条有效指令）
FROM python:3.12-slim

# ② 元信息
LABEL maintainer="zhangsan@example.com"

# ③ 构建过程
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .

# ④ 启动配置
EXPOSE 8000
CMD ["python", "app.py"]
```



组成部分	常用指令
基础镜像	FROM
元信息	LABEL
构建过程	RUN COPY ADD ENV WORKDIR
启动配置	CMD ENTRYPOINT EXPOSE

难点：镜像分层（Layer）与构建缓存

三句话理解核心机制

① 每层只读

每条增添文件的指令生成一个只读层，镜像是层的堆叠

② 层可缓存

指令及之前的内容没变，直接复用缓存，跳过执行

③ 失效向后传导

某层缓存未命中，其后所有层全部重建

两条编写原则

原则一：按变化频率从低到高排列指令

```
# ❌ 反面教材：代码改一行，依赖重装一遍
COPY . .
RUN pip install -r requirements.txt

# ✅ 正确：依赖清单很少变 → 缓存命中率高
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . . # 业务代码天天变 → 放最后
```



原则二：能合并的 RUN 尽量合并

```
# ❌ 三层，中间层残留缓存垃圾
RUN apt-get update
RUN apt-get install -y curl
RUN apt-get clean

# ✅ 一层搞定，顺手清理
RUN apt-get update && apt-get install -y curl && rm -rf /var/lib/apt/lists/*
```



第二部分：常用指令（重点）

🔥 必会：FROM / RUN / COPY / CMD / ENTRYPOINT / EXPOSE

📁 常用：WORKDIR / ENV / ARG / ADD / LABEL / USER / VOLUME ...

FROM —— 指定基础镜像

```
FROM <镜像名>[:<标签>]
```

```
FROM python:3.12-slim
```

```
FROM ubuntu:22.04
```

```
FROM scratch # 空镜像，用于静态编译的二进制（伏笔：实践环节详解）
```



- 必须是 Dockerfile 中**第一条有效指令**（ARG 可在其之前）
- 所有后续操作都在基础镜像之上叠加
- 推荐官方镜像 + 小体积变体（alpine、slim）
- 可多次出现 → **多阶段构建**（见实践环节）

RUN — 构建时执行命令

```
# shell 形式 (默认 /bin/sh -c)
```

```
RUN pip install flask
```

```
# exec 形式 (JSON 数组, 不经过 shell)
```

```
RUN ["pip", "install", "flask"]
```



- 在**构建镜像时**执行，会创建新的镜像层
- 每个 RUN 增加一层 → 尽量合并

⚠ 高频考点: **RUN** 是构建时执行, **CMD** 是容器启动时执行

COPY 与 ADD -- 复制文件

```
COPY app.py /app/          # 复制单个文件
COPY . /app                # 复制构建上下文全部文件（受 .dockerignore 控制）
COPY --chown=www:www . /app # 复制时指定属主

ADD archive.tar.gz /opt/    # ADD 独有：自动解压本地 tar 包
ADD https://example.com/a.zip /tmp/ # ADD 独有：URL 下载（不推荐）
```



- 两者都会创建新镜像层
- COPY 检查文件 Checksum，内容变化 → 缓存失效

✅ 最佳实践：优先用 **COPY**，只有需要"自动解压"时才用 ADD

CMD 与 ENTRYPOINT（重点中的难点）

对比项	CMD	ENTRYPOINT
作用	提供默认启动命令	定义固定入口程序
<code>docker run img echo hi</code>	被替换为 <code>echo hi</code>	变成 <code>entrypoint + echo hi</code>
覆盖方式	命令行参数直接覆盖	需 <code>--entrypoint</code> 才能覆盖
典型用法	指定默认参数	指定可执行程序

经典组合：入口固定、参数可换

```
ENTRYPOINT ["ping"]  
CMD ["localhost"]  
# docker run img          → ping localhost  
# docker run img 8.8.8.8 → ping 8.8.8.8
```



- 一个 Dockerfile 多个 CMD，只有最后一个生效
- 推荐 **exec 形式** (JSON 数组)：信号正确传递，容器可正常停止

EXPOSE / ENV / ARG / WORKDIR

EXPOSE -- 声明端口（仅元数据）

```
EXPOSE 8080      # 只是"声明/文档", 真正映射靠 docker run -p 8080:8080
```



ENV vs ARG -- 变量的生命周期

```
ENV APP_ENV=production  # 构建期和运行期都存在
ARG VERSION=1.0         # 只在构建期存在, 构建完消失
```



```
docker build --build-arg VERSION=2.0 .  # 构建时覆盖 ARG
```



WORKDIR -- 设置工作目录

```
WORKDIR /app      # 相当于 mkdir + cd
# ❌ 别用 RUN cd /app: 每条 RUN 在独立层执行, cd 对后续指令无效
```



其他常用指令速查

指令	作用	示例
<code>LABEL</code>	镜像元数据	<code>LABEL version="1.0"</code>
<code>USER</code>	指定运行用户（安全加固）	<code>USER nginx</code>
<code>VOLUME</code>	声明数据卷挂载点	<code>VOLUME /data</code>
<code>HEALTHCHECK</code>	容器健康检查	<code>HEALTHCHECK CMD curl -f http://localhost/ exit 1</code>
<code>ONBUILD</code>	为子镜像设置触发器	<code>ONBUILD COPY . /app</code>
<code>SHELL</code>	更换默认 shell	<code>SHELL ["/bin/bash", "-c"]</code>

第三部分：实践教学

编写 Dockerfile，用 docker build 创建镜像

官方 hello-world 镜像剖析

官方资源

- Docker Hub: https://hub.docker.com/_/hello-world
- GitHub 仓库: <https://github.com/docker-library/hello-world>

Dockerfile 全文 -- 只有三行, Docker 世界最小的镜像之一

```
FROM scratch
COPY hello /
CMD ["/hello"]
```



指令

含义

FROM scratch

完全为空的基础镜像：无系统文件、无 shell、无库

COPY hello /

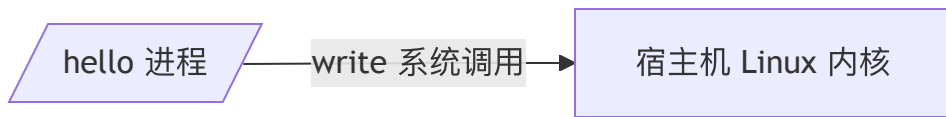
拷入静态编译的 C 可执行文件 (gcc -static)

为什么空镜像 scratch 能运行 hello?

关键认知：容器不是虚拟机

- 虚拟机：带完整 Guest OS + 自己的内核 → 镜像以 GB 计
- 容器：宿主机上的普通进程（namespace 隔离 + cgroups 限资源）
- 所有容器共用宿主机的 Linux 内核，内核不在镜像里

scratch 的"空" = 用户空间为空，系统调用直接找宿主机内核



前提：hello 是静态编译的

	动态链接	静态链接 (hello)
依赖	ld-linux.so + libc.so.6 等	全部代码打包进二进制
在 scratch 里	❌ 启动即报错	✅ 直接运行

```
ldd hello      # not a dynamic executable
ldd /bin/ls    # 列出一串 .so — 没有这些库，ls 根本起不来
```



实战：Flask 应用五步构建

第 1 步：准备文件

```
demo/
├── Dockerfile
├── requirements.txt    # flask==3.0.3
└── app.py             # Flask Web 应用, 监听 5000 端口
```



第 2 步：编写 Dockerfile（缓存原则落地）

```
FROM python:3.12-slim
LABEL maintainer="teacher@example.com"
WORKDIR /app
COPY requirements.txt .                # 先依赖清单
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .                         # 后业务代码
EXPOSE 5000
CMD ["python", "app.py"]
```



实战：构建、验证、体验缓存

第 3 步：构建

```
cd demo
docker build -t flask-demo:1.0 .
# -t: 打标签 (名字:版本)    .: 构建上下文
```



第 4 步：验证

```
docker images          # 查看镜像
docker run -d -p 5000:5000 flask-demo:1.0 # 启动容器、映射端口
curl http://localhost:5000 # Hello, Docker!
```



第 5 步：体验缓存（演示亮点） ✨

```
# 修改 app.py 一行代码，重新构建
docker build -t flask-demo:1.0 .
# 观察输出：pip install 步骤显示 CACHED，秒级完成！
```



进阶：多阶段构建

以 Go 为例：编译期需要完整 SDK（约 1GB），运行期只要一个二进制

```
# 第一阶段：编译
FROM golang:1.22 AS builder
WORKDIR /src
COPY . .
RUN go build -o /bin/server .

# 第二阶段：运行（只拷入编译产物）
FROM scratch
COPY --from=builder /bin/server /server
EXPOSE 8080
ENTRYPOINT ["/server"]
```



最终镜像：约 1GB → 几 MB

- `AS builder` 给阶段命名
- `COPY --from=builder` 跨阶段复制产物
- 第二阶段敢用 `scratch`，前提正是：**静态编译 + 共享宿主机内核**

docker build 常用参数

参数

作用

`-t, --tag`

指定镜像名称和标签, 格式 `name:tag`

`-f`

指定 Dockerfile 路径和名称

`--no-cache`

构建时不使用缓存

`--build-arg`

设置构建时变量 (对应 ARG)

`-q, --quiet`

静默模式, 只输出镜像 ID

`-m, --memory`

设置构建内存上限

```
docker build -t web:latest .  
docker image build -t web:latest . # 等价写法
```



总结

方面	要点
基本结构	FROM → 元信息 → 构建过程 → 启动命令
分层原理	FROM/RUN/COPY 建层，其余记元数据；缓存失效向后传导
四对区别	RUN/CMD · CMD/ENTRYPOINT · COPY/ADD · ENV/ARG
构建命令	<code>docker build -t 名字:标签 .</code> → <code>docker run -p 主机端口:容器端口 镜像</code>
最佳实践	合并 RUN · 不变指令放前面 · 小体积基础镜像 · exec 形式 · 静态编译配 scratch

课后作业：把自己的课程设计项目写成 Dockerfile，下节课验收构建

谢谢大家

Q & A